

SupplyGuard AI

Complete Code Walkthrough and Engineering Study Guide

A method-by-method guide to the finalized local RAG, structured-data, MCP, memory, reliability, and evaluation implementation.

Prepared from the finalized source code | September 2026

Contents

1. System mental model	4
2. Repository map	4
Application entry and interface	4
Core AI and business flow	4
Infrastructure boundaries	5
Ingestion and evaluation	5
Operational files	5
3. How startup works	5
Host and Docker startup	5
main.lifespan(app)	6
4. Ingestion end to end	6
Current flow	6
PDF parsing	6
Chunk creation	7
Embedding and storage	7
FEMA CSV import	7
Adding a new PDF	7
Adding a new CSV	8
5. Why fixed overlap instead of fully structure-aware chunking?	8
6. Streamlit and API connection	8
ui/app.py	8
api/routes.py	8
Pydantic contracts	9
7. Conversational context and memory	9
8. Orchestrator: the central method	9
9. Intent routing	10
10. Document RAG route	10
Hybrid retrieval	10
Reciprocal-rank fusion	10
Reliability before generation	11
Grounded generation	11
11. Structured analytics route	11
12. MCP tool route	11
Server methods	11
Client method	12
13. Data-store responsibilities	12
MongoDB collections	12

Qdrant	12
14. Logging, metrics, health, and audit	12
15. Feedback and RLHF	13
16. Evaluation techniques	13
Chunk metrics	13
Generation metrics	13
17. Edge cases handled	13
Input and security	13
Documents and ingestion	14
Retrieval and generation	14
Analytics and tools	14
Memory and feedback	14
18. Libraries and why	15
19. Test coverage	15
20. Practical extension playbooks	15
Add a new document format	15
Add a structured question	16
Add an MCP tool	16
Change the embedding model	16
Debug a poor answer	16
21. Limitations and next steps	16
22. Presentation checklist	17
Appendix A. Essential commands	17
Appendix B. Method quick reference	17

This report explains the finalized project end to end: how the Streamlit interface calls FastAPI, how the API starts, how documents and CSV records are ingested, how the router selects RAG, MongoDB analytics, or MCP, how Ollama and Qdrant are used, how memory and reliability work, and how to extend and debug the system.

Project: [/Users/akshitarastogi/Documents/Codex/2026-09-09/si/outputs/supplyguard-ai](#)

The central idea is simple: SupplyGuard AI is not one large agent. It is a small orchestrated system with three explicit answer paths. The language model writes grounded document answers, MongoDB computes structured answers, and an independent MCP server owns external-tool operations.

1. System mental model

```
User -> Streamlit -> POST /v1/ask -> authentication and validation
      -> session memory -> guarded hybrid intent router
          -> DOCUMENT: Ollama embedding -> Qdrant + Mongo lexical -> RRF
              -> threshold/conflict checks -> grounded Ollama answer
          -> ANALYTICS: server-built MongoDB aggregation -> computed answer
          -> LIVE_RISK: MCP client -> FastMCP server -> FEMA CSV
      -> typed answer + citations + trace + warnings
      -> MongoDB audit and session-turn storage -> Streamlit display
```

The system separates responsibilities deliberately:

- Ollama generates language and vectors, but it does not calculate CSV totals.
- MongoDB stores records, metadata, feedback, memory, and audit events.
- Qdrant performs vector similarity search with tenant filters.
- The MCP server validates tool inputs and owns FEMA tool behavior.
- FastAPI defines the HTTP boundary and coordinates dependencies.
- Streamlit presents results but contains no retrieval or computation logic.

This separation makes a failure diagnosable. If intent is wrong, inspect routing. If citations are wrong, inspect parsing, chunks, or retrieval. If citations are correct but prose is weak, inspect generation. If a number is wrong, inspect MongoDB or MCP rather than blaming the LLM.

2. Repository map

Application entry and interface

- [ui/app.py](#) builds the Streamlit UI, maintains browser-session state, calls API endpoints, and renders evidence, warnings, traces, feedback, corpus details, and evaluation data.
- [src/supplyguard/main.py](#) creates the FastAPI application and shared infrastructure clients.
- [src/supplyguard/api/routes.py](#) defines authentication, question, feedback, health, readiness, and metrics endpoints.
- [src/supplyguard/domain/models.py](#) defines the typed request, response, citation, feedback, and intent contracts.

Core AI and business flow

- [services/orchestrator.py](#) coordinates memory, routing, retrieval, analytics, MCP, generation, refusal, auditing, and response creation.
- [services/router.py](#) deterministically selects document, structured analytics, or MCP execution.
- [services/memory.py](#) implements bounded tenant-isolated conversational context.
- [services/analytics.py](#) builds safe MongoDB aggregation pipelines over FEMA records.

- `services/contradictions.py` warns when retrieved evidence contains conflicting monetary values.
- `retrieval/hybrid.py` combines lexical and dense retrieval using reciprocal-rank fusion.

Infrastructure boundaries

- `infrastructure/mongo.py` owns native asynchronous PyMongo setup, indexes, and readiness.
- `infrastructure/vector.py` owns Qdrant collections, upserts, filters, search, health, and error normalization.
- `infrastructure/ollama.py` owns local embedding and generation requests, retries, timeouts, validation, concurrency, and health.
- `infrastructure/mcp_client.py` owns Streamable HTTP MCP sessions and tool invocation.
- `mcp_server.py` is the independent FastMCP tools process over the official FEMA snapshot.

Ingestion and evaluation

- `ingestion/models.py` defines page elements and retrieval chunks.
- `ingestion/parsers.py` extracts PDF text, tables, image states, and blank-page states.
- `ingestion/chunker.py` creates fixed overlapping chunks and deterministic IDs.
- `scripts/seed.py` is the offline parse, embed, index, and CSV-import pipeline.
- `scripts/benchmark_chunking.py` compares chunk candidates using measured retrieval proxies.
- `scripts/evaluate.py` calls the live API and writes answer-quality scores.
- `evaluation.py` contains reusable completeness, grounding, citation, and refusal metrics.

Operational files

- `requirements.txt` is the single unpinned dependency file requested for the assignment.
- `Dockerfile` builds the Python image, uses a non-root user, sets `PYTHONPATH=/app/src`, and starts Uvicorn.
- `docker-compose.yml` defines API, UI, MongoDB, Qdrant, and MCP services.
- `.env` and `.env.example` configure endpoints, model names, thresholds, memory limits, and demo API roles.
- `pytest.ini` enables async tests and imports from `src` without a pyproject file.
- `.github/workflows/ci.yml` installs dependencies, runs Ruff and tests, and builds Docker.
- `evaluation.json` contains 13 expected-behavior cases.
- `chunking_benchmark.csv` stores measured results for all three chunk candidates.

3. How startup works

Host and Docker startup

```
ollama serve
ollama pull mistral:instruct
ollama pull nomic-embed-text
cp .env.example .env
docker compose up --build -d
docker compose exec api python scripts/seed.py
```

Ollama remains on the host so the existing local installation and hardware are used. Containers reach it through `host.docker.internal`. Compose starts MongoDB, Qdrant, the MCP tools process, FastAPI, and Streamlit.

main.lifespan(app)

Uvicorn imports `supplyguard.main:app`. FastAPI enters `lifespan()` before serving traffic:

1. `configure_logging()` installs structured JSON logging.
2. `get_settings()` reads `.env` once and caches the typed settings.
3. `Mongo(settings)` creates a lazy native asynchronous PyMongo client.
4. `OllamaClient(...)` creates one pooled async HTTP client and a concurrency semaphore.
5. `VectorStore(...)` creates the asynchronous Qdrant client.
6. `MCPGateway(...)` stores the MCP endpoint.
7. MongoDB is pinged. When ready, application indexes are created.
8. The four dependencies and settings are passed into `Orchestrator`.
9. A startup event is logged.

On shutdown, Ollama HTTP connections, Qdrant connections, and the asynchronous MongoDB client are awaited and closed. Uvicorn uses two workers, so each process owns its own event-loop-bound clients.

Ingestion is intentionally not performed during API startup. Parsing 603 pages and embedding thousands of chunks would make every restart slow and could create startup races. It is an explicit offline command.

4. Ingestion end to end

Current flow

```
scripts/seed.py main()
-> find data/documents/public/*.pdf
-> parse_document(path)
-> parse_pdf(path) -> PageElement objects
-> chunk_element(element, demo-corp) -> Chunk objects
-> upsert canonical chunks in MongoDB
-> OllamaClient.embed_batched()
-> VectorStore.ensure_collection(actual vector dimension)
-> Qdrant upsert batches of 128
-> read and type FEMA CSV rows
-> upsert structured records in MongoDB
-> write ingestion_runs summary
```

PDF parsing

`parse_document(path)` is the dispatch method. PDF files go to `parse_pdf`; Markdown and text go to `parse_markdown`; unknown extensions raise `UnsupportedDocument`.

`parse_pdf(path)` uses PyMuPDF because it provides fast page access plus text, table, and image inspection. For every page it:

- rejects encrypted PDFs that require a password;
- extracts sorted reading-order text;
- finds tables and serializes rows separately;
- writes missing table cells as `<NULL>` instead of dropping their position;
- counts page images;
- marks image-only pages as `ocr_required`;
- marks non-image pages with no extractable text as `blank_page`;
- preserves source path, one-based page number, element type, and metadata.

Text and tables become separate `PageElement` objects. This prevents a dense table from distorting nearby narrative text and gives citations a meaningful `text` or `table` locator.

The code identifies pages that need OCR but does not run OCR. That is an honest limitation: detecting an unread page is better than silently calling ingestion successful.

Chunk creation

`chunk_element(element, tenant_id, size=160, overlap=30)` splits the element by words. It never crosses a page-element boundary. For each chunk it builds:

- `source_id`: first 16 characters of a SHA-256 hash of the source path;
- `locator`: page, element kind, and chunk ordinal;
- `point_id`: UUID5 over tenant, source, and locator;
- `content_hash`: SHA-256 of the chunk body;
- title, text, ordinal, and metadata.

The tenant participates in the point UUID so identical sources cannot collide across tenants. Stable IDs make repeat seeding idempotent: MongoDB and Qdrant update the same logical records.

Embedding and storage

`OllamaClient.embed_batched()` sends 16 chunks at a time to `nomic-embed-text`. Batching bounds memory and model pressure. `embeddings()` verifies that the model returned exactly one vector per input.

The seed script reads the first returned vector length and passes it to `VectorStore.ensure_collection()`. This avoids duplicating `768` as a fragile constant. A new embedding model should use a new collection name and a full reindex because different vector spaces are incompatible.

Qdrant points are written in batches of 128. MongoDB is the canonical text/metadata store; Qdrant contains the vector and a retrievable payload.

FEMA CSV import

The official CSV is read with `csv.DictReader`. Each row receives tenant ID, record type, and stable record ID. `disasterNumber` and `fyDeclared` are converted to integers. Rows are upserted using tenant and record ID, so repeated seeding does not duplicate them.

Row-by-row upsert favors clarity for this assessment. A production version should validate required headers first, quarantine malformed rows, convert all dates and booleans explicitly, and use `bulk_write` batches.

Adding a new PDF

1. Place the official PDF in `data/documents/public/`.
2. Record source URL, edition/date, SHA-256, structure, and known risks in `docs/DATA_MANIFEST.md`.
3. Inspect encryption and whether pages are text, scanned, or mixed.
4. Add at least one expected-answer question to `evaluation.json`.
5. Re-run `scripts/benchmark_chunking.py`; a different document style can change the best size.
6. Run `docker compose exec api python scripts/seed.py`.
7. Inspect ingestion counts and parser failures.
8. Run the evaluation and manually inspect page citations.

A successful chunk count does not prove correct reading order or table extraction. Citation inspection is required.

Adding a new CSV

Create an explicit record type and stable record ID. Validate headers before looping. Convert types instead of storing everything as strings. Add appropriate MongoDB indexes. Then add an allowlisted analytics operation or a new MCP tool. Never accept a raw MongoDB pipeline from user text.

5. Why fixed overlap instead of fully structure-aware chunking?

The final default is 160 words with 30 words overlap, producing a 130-word step. It is page-aware and element-aware, but it is not heading-tree semantic chunking.

The source PDFs come from different publishers and contain inconsistent heading encodings, long control tables, annexes, figures, and pages whose visual hierarchy is not reliably represented in extracted text. A strategy based only on font size, bookmarks, or layout blocks can create false structure. Fixed windows are deterministic, easy to reproduce, and easy to compare within a three-hour assignment.

The project did not merely choose a familiar number. It measured three candidates across the real PDFs and four golden document questions:

Candidate	Chunks	Recall@6	Hit@6	MRR@6	Context words
160/30	2,198	74.6%	100%	0.413	749
240/40	1,562	70.5%	100%	0.396	1,067
400/60	1,114	74.1%	100%	0.479	1,434

The 160/30 variant won on concept recall and smallest top-six evidence packet. It creates more vectors and has slightly higher lexical latency, but retrieval quality is more important than a small storage saving for this corpus. The 400-word version ranked the first relevant item best but sent almost twice as much text to generation, increasing distraction and context cost.

A mature structure-aware version would detect headings, lists, captions, annex boundaries, and parent-child relationships. It could retrieve small children and send their larger parent section to generation. It should only replace the current method if it wins the same measured evaluation, especially after OCR and layout analysis are available.

6. Streamlit and API connection

ui/app.py

Streamlit reads `API_BASE_URL` and `API_KEY` from environment variables. It creates a UUID `conversation_id` and `messages` list in `st.session_state`. The `New chat` button replaces the ID and clears visible history.

The sidebar calls `GET /health/ready` with a short timeout. The Ask tab posts this body to FastAPI:

```
{"tenant_id": "demo-corp", "question": "...", "conversation_id": "uuid"}
```

The API key is sent as `X-API-Key`. The UI then displays intent, evidence confidence, citation count, answer text, warnings, evidence excerpts, and tool execution trace. Feedback posts request ID, helpful/not-helpful, and optional reason to `/v1/feedback`.

The UI is intentionally a thin client. It does not import the retriever, open MongoDB, or calculate results. That keeps API and UI behavior consistent.

api/routes.py

- `actor()` maps the API key to a configured role or raises HTTP 401.

- `ask()` receives a validated `AskRequest` and delegates to `app.state.orchestrator.ask()`.
- `feedback()` upserts one response signal by request ID and states that no model update occurs.
- `live()` proves that the process responds.
- `ready()` checks MongoDB, Qdrant, and Ollama independently and returns HTTP 503 when degraded.
- `metrics()` exposes Prometheus counters and histograms.

Pydantic contracts

`AskRequest` bounds question length, restricts tenant identifiers, validates conversation identifiers, and restricts filters to scalar values. Its validator normalizes whitespace and rejects invisible control characters.

`AskResponse` guarantees that every execution path returns request ID, answer, intent, confidence, citations, tool trace, warnings, and creation time. A uniform response makes UI rendering and auditing reliable.

7. Conversational context and memory

```
tenant_id + conversation_id
-> load at most 6 newest turns
-> is current question a short referential follow-up?
    no: current question only
    yes: bounded context packet, maximum 4,000 characters
-> normal route and answer
-> save turn
-> MongoDB TTL expires it after 24 hours
```

`is_follow_up()` looks for short referential phrases such as `What about`, `And`, `that`, or `previous`. A clear new question does not inherit stale context.

`load_recent_turns()` filters both tenant and conversation ID, sorts newest first, limits to six, then reverses into chronological order. Database failure returns an empty list so memory cannot take down answering.

`build_context()` labels history as context only, not evidence. It truncates the oldest portion when the 4,000-character budget is exceeded while preserving the current question and newest context.

`store_turn()` writes the original question and final answer. Memory usage is included in the tool trace and audit event.

This is session context, not permanent model memory. The model weights do not change. Conversation text can resolve missing intent and parameters, but it cannot replace retrieved document citations. Production should also expose explicit deletion and retention controls.

8. Orchestrator: the central method

`Orchestrator.ask(request, actor)` is called by the question route. Its order matters:

1. Generate a request UUID.
2. Load recent conversation turns.
3. Attach them only if the question is a follow-up.
4. Classify intent from the contextualized question.
5. Add a memory trace when a conversation ID exists.
6. Execute exactly one answer path.
7. Construct the typed `AskResponse`.
8. Insert a full audit event.

9. Store the session turn.

10. Return the response to the route and UI.

Dependencies are passed into the constructor rather than imported globally. This is dependency injection: tests can provide fakes, and infrastructure can be replaced without rewriting business flow.

9. Intent routing

Routing uses a guarded hybrid instead of FuzzyWuzzy or substring matching alone:

- `explicit_classify()` protects direct requests such as `use the FEMA tool`, `MCP tool`, or `live disaster risk` and selects `LIVE_RISK` without a probabilistic call.
- Other questions call `OllamaClient.classify_intent()` using local `mistral:instruct`, temperature zero, a 20-token limit, and a JSON schema restricted to `document`, `analytics`, or `live_risk`.
- Invalid JSON, an unknown enum, timeout, or Ollama failure activates `classify()`, the deterministic keyword fallback.

The LLM handles paraphrases better than fuzzy string similarity because intent is semantic, not spelling similarity. Explicit-tool protection prevents the model from overriding a direct instruction, and the fallback keeps the system operational when local inference is unavailable. The tool trace records `ollama`, `explicit_rule`, or `rule_fallback`, so routing remains inspectable. The cost is one additional local model call for non-explicit questions; an intent confusion matrix and routing latency should be measured.

Memory enables `What about 2024?` to reuse an earlier FEMA-tool route. Current-turn state/year values win; history fills only omitted parameters.

10. Document RAG route

Hybrid retrieval

`retrieve()` performs two searches:

```
MongoDB tenant/filter chunks -> lexical_score -> top 20
Question -> Ollama embedding -> Qdrant cosine search -> top 20
Both rank lists -> reciprocal-rank fusion -> top 6 citations
```

`tokens()` produces lowercase alphanumeric tokens. `lexical_score()` uses term-frequency cosine similarity. This protects exact acronyms, numbers, control IDs, and rare procurement terms. It is intentionally inspectable but is not BM25.

The lexical path reads tenant/filter-matching canonical chunks from MongoDB with a safety cap of 10,000. The cap covers the current 2,198 chunks. A much larger corpus should use a proper BM25 index instead of application-memory ranking.

The dense path sends the question to `nomic-embed-text`. Qdrant performs cosine search with a mandatory tenant payload condition plus optional metadata filters. Dense vectors capture semantic paraphrases that exact terms miss.

Reciprocal-rank fusion

Dense and lexical scores use different numeric scales, so raw scores are not added. RRF contributes $1 / (60 + \text{rank})$ for each list. A chunk highly ranked by both paths gets both contributions. The constant 60 reduces sensitivity to small rank movements and avoids score calibration.

The final score is normalized against the maximum score possible at rank one in both lists. Six citations balance cross-document coverage against the limited context of a local model.

Reliability before generation

The orchestrator averages the first three citation scores as a retrieval-confidence heuristic. If no citations exist or confidence is below 0.35, it refuses: it does not ask Mistral to answer from general memory.

`contradictions.detect()` scans evidence for different USD/dollar values associated with approval or monetary rules. It emits a warning rather than arbitrarily selecting a value.

Embedding or vector-store failure becomes a safe refusal. The displayed confidence is retrieval confidence, not a calibrated probability of factual correctness.

Grounded generation

The top evidence is formatted as labeled source blocks such as `[S1] title (page locator)`. The whole packet is limited to 12,000 characters.

`OllamaClient.generate()` tells `mistral:instruct` that evidence is untrusted quoted data, not instructions; answers must come only from evidence; insufficient or conflicting evidence must be acknowledged; numeric units must remain exact; and sources should use `[S#]` labels.

Generation uses temperature zero, at most 500 predicted tokens, connection and response timeouts, one transient retry, and a semaphore allowing four simultaneous local-model requests. Empty output raises `OllamaUnavailable`. The fallback returns retrieved citations with a generation-failure warning, never an ungrounded answer.

11. Structured analytics route

`answer_analytics()` does not use the LLM for arithmetic. It selects an allowlisted grouping field: state, incident type, fiscal year, or region. It optionally extracts a four-digit year and builds this server-owned pipeline:

```
$match tenant + record_type + optional year
$group by allowlisted field and count
$sort count descending, identifier ascending
$limit 1 or 5
```

The deterministic secondary sort makes ties repeatable. The trace records dataset, measure, group field, year, and returned-row count. No matching rows produce an explicit message.

User text cannot supply a collection, MongoDB operator, JavaScript expression, or arbitrary pipeline. This prevents NoSQL injection and guarantees that final numbers come from real computation.

12. MCP tool route

MCP is used as a genuine process boundary. FastAPI is a client; `mcp_server.py` is an independent Streamable HTTP server that reads the immutable FEMA CSV.

Server methods

`rows()` loads the CSV once per MCP process using `lru_cache(maxsize=1)`. This avoids a 70,402-row disk scan for every call.

`lookup_disasters(state, incident_type, limit)` validates an exact two-letter state, clamps the result limit to 1-50, filters optional incident type, sorts newest first, and returns JSON containing source, total match count, and bounded results.

`disaster_statistics(state, year)` validates an optional state and a FEMA-range year, filters rows, counts matches, groups incident types with `Counter`, and returns source, applied filters, total, and breakdown.

Client method

`MCPGateway.call(name, arguments)` opens Streamable HTTP, creates and initializes `ClientSession`, invokes the tool, rejects MCP errors or missing content, and parses the first content block as JSON. Transport, session, tool, and parsing failures become `MCPToolError`.

On failure, the orchestrator returns zero confidence, a failed tool trace, and a dependency warning. It never fabricates a FEMA count. On success, the trace contains the tool name, arguments, source, and result count, meeting the assessment requirement to show the tool output used.

13. Data-store responsibilities

MongoDB collections

- `chunks`: canonical chunk text and metadata; unique (`tenant_id`, `point_id`).
- `records`: typed FEMA records used for computation.
- `audit_events`: questions, actors, routes, responses, traces, and memory use; expires after 90 days.
- `feedback`: one updatable quality signal per request ID.
- `conversation_turns`: tenant/chat-specific session history; expires after 24 hours.
- `ingestion_runs`: document count, chunk count, embedding model, and parser failures.

Native asynchronous PyMongo is used instead of deprecated Motor. MongoDB fits heterogeneous JSON-like records, audit events, feedback, and memory.

Qdrant

Qdrant stores dense vectors plus retrieval payloads. It provides persistent cosine search and server-side metadata filters in Docker. Every search requires tenant ID.

FAISS would be fast but requires application code for persistence, metadata filtering, and concurrent access. Chroma is convenient, but Qdrant exposes collection and payload behavior clearly. Local MongoDB Community is not presented as Atlas Vector Search.

14. Logging, metrics, health, and audit

`configure_logging()` configures structlog JSON with timestamp, level, and context variables.

`RequestContextMiddleware.dispatch()` accepts or creates `X-Request-ID`, clears old context, binds request ID/method/path, starts a timer, calls the endpoint, returns the ID header, increments a Prometheus counter, observes latency, logs completion, and clears context in `finally` even after errors.

- `/health/live` checks process liveness.
- `/health/ready` checks MongoDB, Qdrant, and Ollama separately.
- `/metrics` exposes request count and latency histograms.

The audit event explains a decision after the fact. Logs answer what happened operationally. Tool traces answer how the response was produced. Citations answer what supports a document claim. They complement each other.

Unexpected exceptions are logged with path and type, while the client receives a generic 500 body that does not expose secrets or stack details.

15. Feedback and RLHF

The UI records Helpful or Not helpful plus an optional reason. The API upserts it by request ID. No prompt, policy, reward model, or model weight changes automatically. This is product-quality feedback, not reinforcement learning and not RLHF.

A future reviewed loop would join feedback with citations and traces, classify routing/retrieval/generation/tool failures, validate labels, add confirmed failures to the golden set, change one component, and run regressions. Training automatically on raw votes would introduce noise and abuse risk.

16. Evaluation techniques

`evaluation.json` includes four document, two structured, one MCP, one refusal, and five bonus cases covering cross-document comparison, tables, acronyms, adversarial behavior, and tenant isolation.

Chunk metrics

`benchmark_chunking.py` reports:

- concept recall@6: expected-answer concepts recovered in top six chunks;
- hit rate@6: whether each question retrieves at least one relevant chunk;
- MRR@6: how early the first relevant result appears;
- lexical ranking latency;
- top-six context words;
- total chunk/vector count.

These are retrieval proxies, not final answer accuracy.

Generation metrics

`evaluate.py` calls the live API and uses `evaluation.answer_metrics()`:

- Completeness: expected concept words found in the answer.
- Groundedness: meaningful answer terms also found in returned evidence.
- Citation coverage: distinct `[S#]` labels divided by returned citations.
- Refusal correctness: whether refusal behavior matches the case category.

Document cases pass when completeness and groundedness are at least 0.50. Other routes require completeness and correct refusal behavior. These are transparent lexical proxies; they do not understand every synonym or entailment. Human citation review and numeric exact match should supplement them.

No generation score should be claimed unless the seeded live stack produced it. Availability is not a passing score.

17. Edge cases handled

Input and security

- Missing/invalid API keys return HTTP 401.
- Bad tenant and conversation identifiers fail Pydantic validation.
- Question length and filter value types are bounded.
- Control characters are rejected.

- MongoDB aggregation is server-built and allowlisted.
- MongoDB and Qdrant operations always apply tenant scope.
- Retrieved documents are labelled untrusted evidence, not prompt instructions.
- Generic unexpected-error responses avoid leaking internal details.

Documents and ingestion

- Password-protected PDFs are rejected and recorded.
- Missing table cells retain their location as `<NULL>`.
- Image-only pages are marked for OCR.
- Blank/non-extractable pages are explicitly represented.
- Unknown extensions raise a typed error.
- Stable hashes and UUIDs make ingestion idempotent.
- Embedding count mismatch is rejected.
- Collection dimension is derived from the real embedding response.

Retrieval and generation

- Empty query token sets return zero lexical score.
- Weak or missing evidence produces refusal.
- Qdrant/Ollama failures do not trigger model-memory answers.
- Conflicting monetary evidence creates a warning.
- Evidence and model output are bounded.
- Transient model connection/timeouts receive one retry.
- Empty generated answers are rejected.

Analytics and tools

- No matching records are explicitly reported.
- Structured answers are computed, not hardcoded.
- MCP state/year inputs are validated.
- Lookup limits are clamped to 1-50.
- MCP transport, tool, empty-content, and JSON failures are normalized.
- Tool failure is exposed in the trace without fabricated data.

Memory and feedback

- Memory is isolated by tenant and conversation.
- Only short referential follow-ups inherit history.
- History is limited by turns and characters.
- Old sessions expire automatically.
- Memory database failure degrades to a standalone question.
- Repeated feedback updates the same request record.

18. Libraries and why

- FastAPI: typed asynchronous API, Pydantic integration, and generated OpenAPI docs.
- Streamlit: fast assessment UI that can clearly expose evidence and traces.
- PyMongo: current official native asynchronous MongoDB driver.
- qdrant-client: persistent local vector search and payload filters.
- httpx: asynchronous pooling, timeouts, retries, and test transports.
- PyMuPDF: efficient page text, table, and image access.
- mcp: real Model Context Protocol server/client with Streamable HTTP.
- structlog: structured JSON and request-scoped context.
- prometheus-client: standard counters and histograms.
- pydantic-settings: typed environment configuration.
- pytest and pytest-asyncio: unit and asynchronous service tests.
- Ruff: fast correctness, import, and style checks.

19. Test coverage

- `test_core.py`: routing, lexical relevance, contradiction warning, request normalization, tenant validation.
- `test_ingestion.py`: mixed PDF tables/images/page locators and deterministic tenant-specific IDs.
- `test_vector.py`: Qdrant tenant isolation.
- `test_analytics.py`: allowlisted pipeline and integer year filter.
- `test_mcp_tools.py`: invalid states/years and real FEMA computation.
- `test_ollama.py`: empty generation and embedding-count validation.
- `test_memory.py`: follow-up context, bounded truncation, no-history behavior, and topic switching.
- `test_evaluation_metrics.py`: completeness, grounding, citations, and refusal scores.

The final audit passed 19 tests, Ruff, Python compilation, dependency consistency, Compose parsing, application import, data schema/count, checksums, and a check that every source/script class and method has documentation.

Full live Docker/Ollama generation was not claimed during finalization because those daemons were unavailable in the authoring environment.

20. Practical extension playbooks

Add a new document format

1. Write a parser that returns `PageElement` objects.
2. Add extension dispatch in `parse_document()`.
3. Preserve source, page, element kind, and parser metadata.
4. Add encrypted/corrupt/blank/mixed-content tests.
5. Benchmark chunks and add golden questions before seeding.

Add a structured question

1. Add only the necessary allowlisted field or operation.
2. Build the MongoDB pipeline on the server.
3. Return a detailed computation trace.
4. Test exact pipeline shape and empty results.
5. Add a numeric exact-match case.

Add an MCP tool

1. Define a typed `@mcp.tool()` function in `mcp_server.py`.
2. Validate and bound every input inside the tool.
3. Include source and applied filters in output.
4. Extend routing and orchestrator argument extraction.
5. Normalize dependency failure and expose output in `tool_trace`.
6. Add direct tool, routing, and failure tests.

Change the embedding model

1. Pull the new local Ollama model.
2. Change `OLLAMA_EMBED_MODEL`.
3. Change `QDRANT_COLLECTION` to a new versioned name.
4. Reindex every document.
5. Confirm returned vector dimension and collection configuration.
6. Re-run retrieval and live answer evaluations.

Debug a poor answer

1. Check response intent.
2. Inspect tool arguments or aggregation trace.
3. For RAG, inspect actual citation pages and scores.
4. Wrong evidence means parsing/chunking/retrieval failure.
5. Correct evidence but wrong prose means generation/prompt failure.
6. Inspect warnings for refusal, contradiction, or dependency failure.
7. Use request ID to correlate JSON logs and the audit record.
8. Add the failure to `evaluation.json` before changing code.

21. Limitations and next steps

- OCR and vision are not implemented; scanned pages are only flagged.
- Fixed chunks are page/element aware but not heading-aware parent-child chunks.
- Lexical retrieval is term-frequency cosine, not BM25.
- The local LLM router can still misclassify domain-overlapping questions; its deterministic fallback is narrower.
- Retrieval confidence is heuristic and uncalibrated.

- Answer metrics are lexical proxies and need reviewed faithfulness evaluation.
- FEMA import is row-by-row rather than batched.
- Demo API keys are not production identity.
- The FEMA snapshot is reproducible but not live truth.

The best next work is measurement-led: label relevant pages, compare hybrid retrieval variants, inspect citation precision, implement OCR, batch ingestion, and add numeric exact match. Fine-tuning is not the first fix because it cannot repair parsing, routing, retrieval, or stale-data failures.

22. Presentation checklist

- Begin with the three answer paths.
- Show a document answer and inspect page citations.
- Show MongoDB computation and its trace.
- Explicitly request the FEMA tool and inspect its output.
- Ask an unsupported question and explain refusal.
- Ask **What about 2024?** after an MCP question to demonstrate session context.
- Explain that chunk size was experimentally changed to 160/30.
- State that feedback is not RLHF and confidence is not factual probability.
- Acknowledge OCR, BM25, routing, and metric limitations clearly.

Appendix A. Essential commands

```
# Models
ollama serve
ollama pull mistral:instruct
ollama pull nomic-embed-text

# Start and seed
cp .env.example .env
docker compose up --build -d
docker compose exec api python scripts/seed.py

# Open
http://localhost:8501
http://localhost:8000/docs

# Validate
pytest -q
ruff check --line-length 100 src tests scripts ui

# Experiments
PYTHONPATH=src python scripts/benchmark_chunking.py
python scripts/evaluate.py

# Health and shutdown
curl http://localhost:8000/health/ready
docker compose down
```

Appendix B. Method quick reference

- **main.lifespan**: creates and closes application dependencies.
- **routes.actor**: authenticates API key and returns its role.

- `routes.ask`: delegates a valid question to the orchestrator.
- `routes.feedback`: upserts explicit evaluation feedback.
- `Mongo.ensure_indexes`: creates uniqueness, lookup, audit, and memory indexes.
- `Orchestrator.ask`: coordinates memory, route, answer, response, and audit.
- `memory.is_follow_up`: decides whether recent context is needed.
- `memory.load_recent_turns`: loads bounded tenant/chat history.
- `memory.build_context`: creates a capped context-only transcript.
- `router.classify_with_llm`: selects intent through guarded local-LLM classification and fallback.
- `OllamaClient.classify_intent`: returns one schema-constrained intent enum.
- `parse_pdf`: extracts page text, tables, images, and empty-page states.
- `chunk_element`: creates overlapping deterministic chunks.
- `OllamaClient.embeddings`: returns validated vectors.
- `OllamaClient.generate`: generates a bounded evidence-only answer.
- `VectorStore.search`: runs tenant-filtered Qdrant cosine search.
- `lexical_score`: calculates term-frequency cosine similarity.
- `retrieve`: fuses lexical and dense ranks into citations.
- `detect`: warns about conflicting retrieved monetary values.
- `answer_analytics`: executes an allowlisted MongoDB aggregation.
- `MCPGateway.call`: opens MCP, invokes a tool, and parses JSON.
- `rows`: caches the immutable FEMA snapshot.
- `lookup_disasters`: returns bounded recent FEMA records.
- `disaster_statistics`: computes FEMA totals and incident breakdowns.
- `answer_metrics`: calculates transparent quality proxies.